BBSNS: Blockchain Forensic Hex Guide

An elegant, step-by-step forensic guide showing how to read, navigate, and slice raw transaction Input bytecode to verify on-chain metadata.

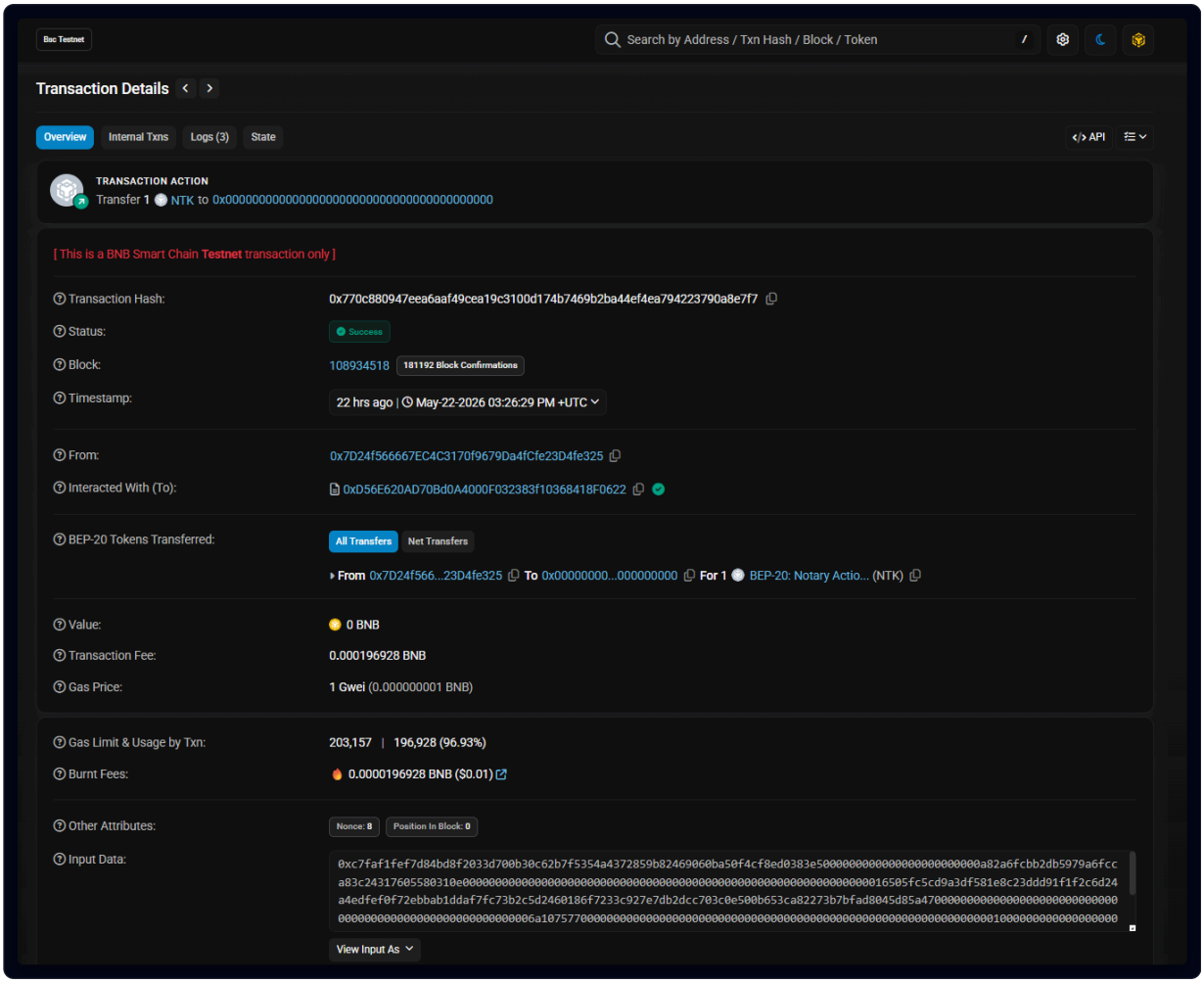
🔐 100% Client-Side Portable & Shareable Guide

This HTML dashboard is built to be **completely offline-portable**. By sharing this folder containing both the screenshots and this HTML file, anyone on any device (Windows, macOS, Linux, Android, iOS) using any browser (Chrome, Edge, Safari, Firefox) can open it and view the exact step-by-step images and textual guides side-by-side without any issues.

Every parameter passed to our Solidity contract during a notarization call is formatted in 32-byte chunks (exactly 64 hex characters long). Slicing these parameters chunk-by-chunk is the absolute, mathematically irrefutable verification of on-chain document notarization.

STEP 1

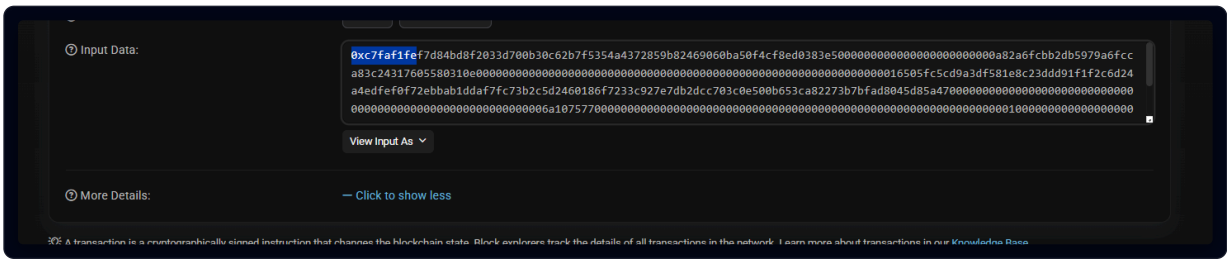
Overview of Transaction Details on BscScan



The starting point of our manual audit. Enter your transaction hash `0x770c880947eea6aaf49cea19c3100d174b7469b2ba44ef4ea794223790a8e7f7` into BscScan. This screen verifies block confirmation, execution success, from/to address records, NTK token burn transfer details, and contains the raw **Input Data** payload box at the bottom.

STEP 2

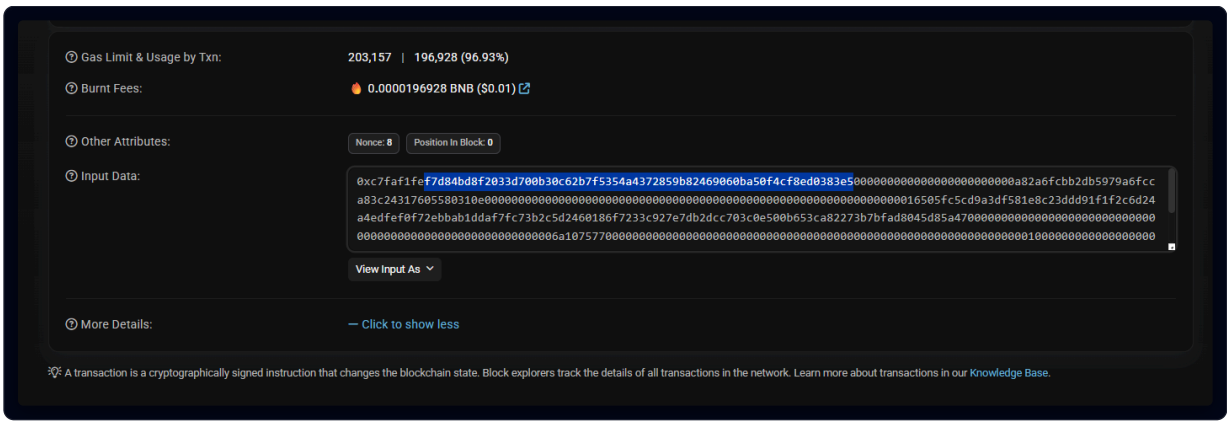
Slicing Slot 0: Function Selector / Method ID



The first 4 bytes (8 hex characters) of the Input Data array: `0xc7faf1fe`. In the EVM, this Method ID acts as a unique signature that tells the smart contract to execute our `recordAction(bytes32,address,uint8,bytes32,bytes32,uint256,uint256,bytes)` function.

STEP 3

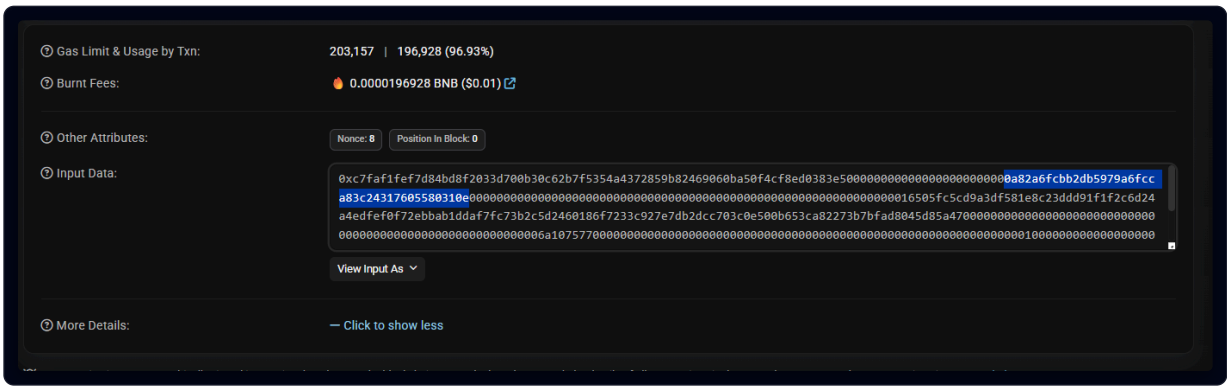
Slicing Slot 1: Document SHA-256 Cryptographic Hash



Slicing the next 32 bytes (64 hex characters): `f7d84bd8f2033d700b30c62b7f5354a4372859b82469060ba50f4cf8ed0383e5`. This represents the immutable digital fingerprint of the uploaded document, proving that the exact file content has been registered on-chain.

STEP 4

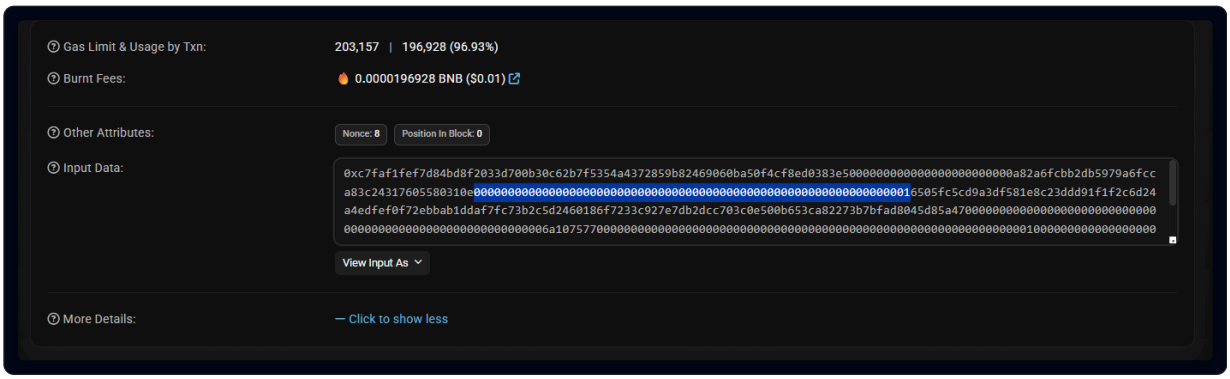
Slicing Slot 2: Owner Wallet Address



Slicing the next 32 bytes (64 hex characters): `000000000000000000000000a82a6fcbb2db5979a6fcca83c2431760558031e`. In Solidity, addresses are 20 bytes long. They are left-padded with 12 bytes (24 zeros) of spacing, leaving the final 40 characters as the owner's unique personal wallet address.

STEP 5

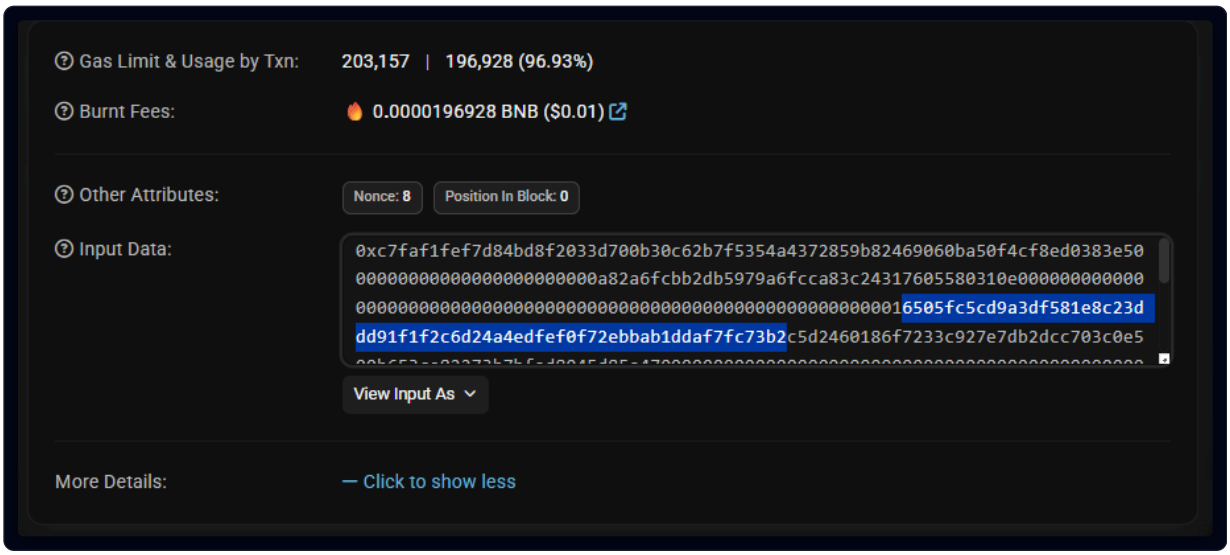
Slicing Slot 3: Notarization Status



Slicing the next 32 bytes (64 hex characters): 0000000000000000000000000000000000000000000000000000000000000001. The decimal value 1 represents **Legally Approved**, permanently setting the document status to active in contract storage (0 = Pending, 2 = Rejected).

STEP 6

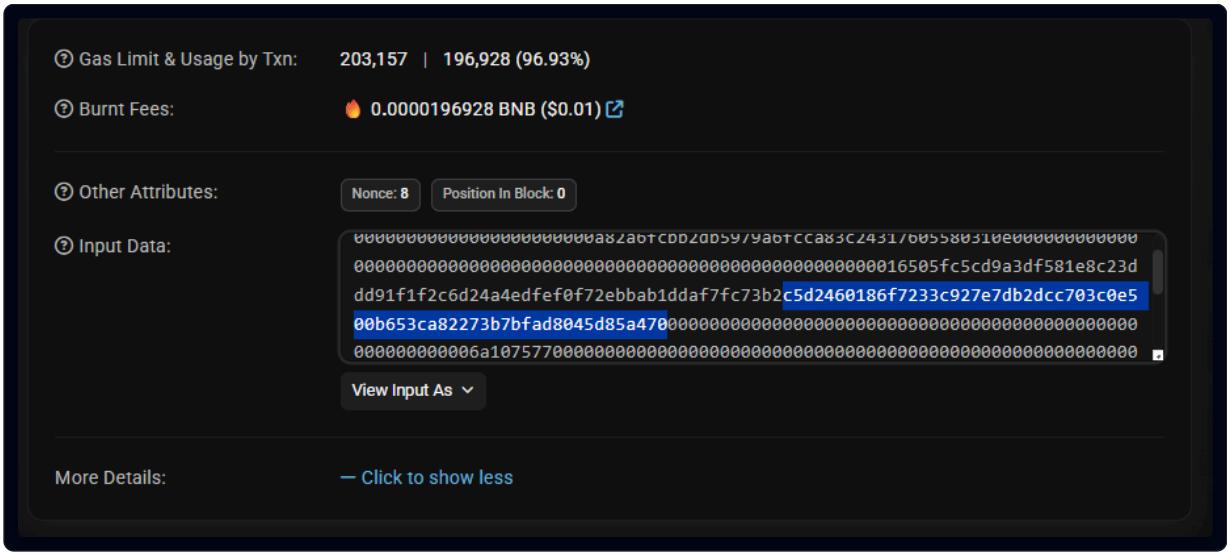
Slicing Slot 4: Notary Summary Hash



Slicing the next 32 bytes (64 hex characters): `6505fc5cd9a3df581e8c23ddd91f1f2c6d24a4edfef0f72ebbab1ddaf7fc73b2`. This is the Keccak-256 hash of the summary description written by the notary during approval, safeguarding the description contents from any retroactive modifications.

STEP 7

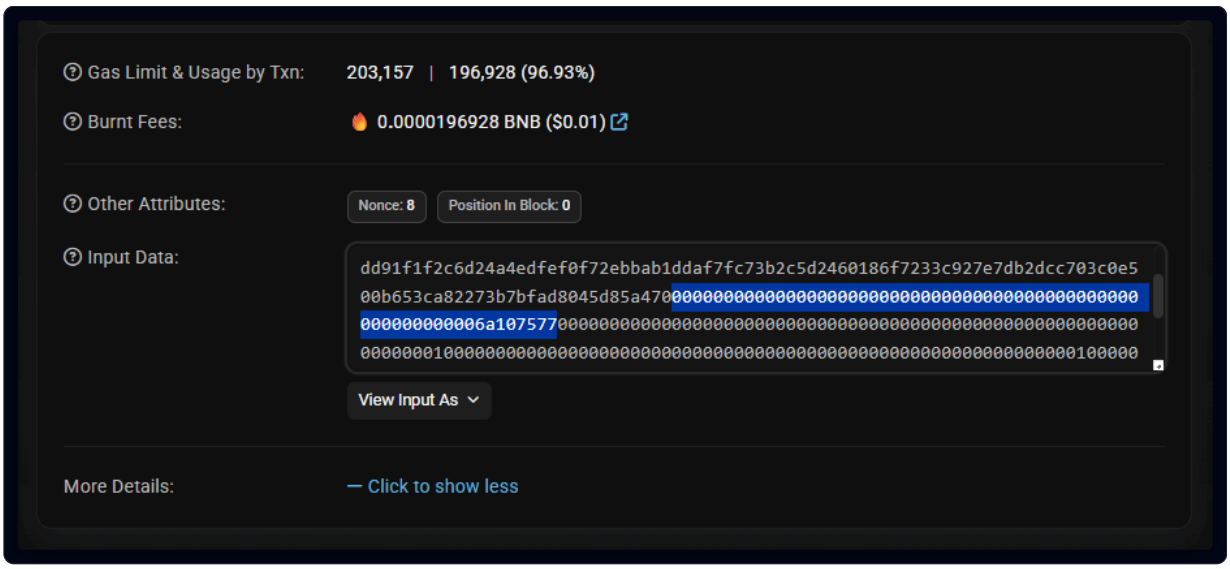
Slicing Slot 5: Rejection Reason Hash (Cryptographic Empty String Hash)



Slicing the next 32 bytes (64 hex characters): `c5d2460186f7233c927e7db2dcc703c0e500b653ca82273b7bfad8045d85a47`. Because Solidity expects a strict, static `bytes32` type for this field, the client cannot pass a null or variable-length empty string. Instead, for approved documents, the client hashes a completely blank string: `keccak256("")`. This outputs this exact cryptographic constant, mathematically proving that no rejection reason exists for this approved record.

STEP 8

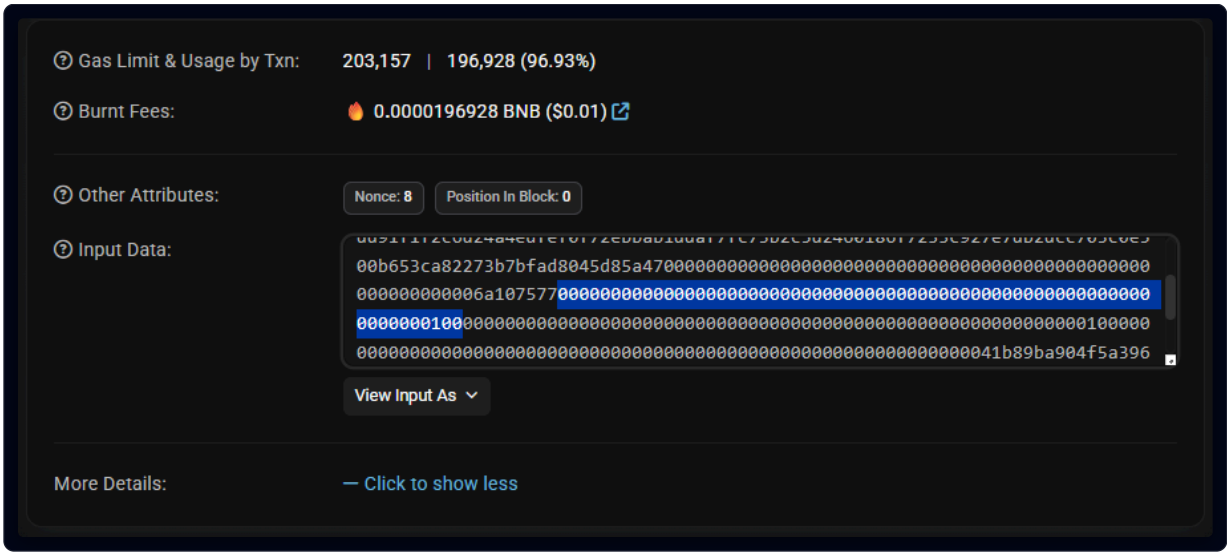
Slicing Slot 6: Signature UNIX Timestamp



Slicing the next 32 bytes (64 hex characters): `000000000000000000000000000000000000000000000000000006a1075770`. Converting the hex value `0x6a107577` to decimal results in `1779512695`, which translates to the exact notarization epoch: **Saturday, May 23, 2026 7:04:55 PM UTC**.

STEP 9

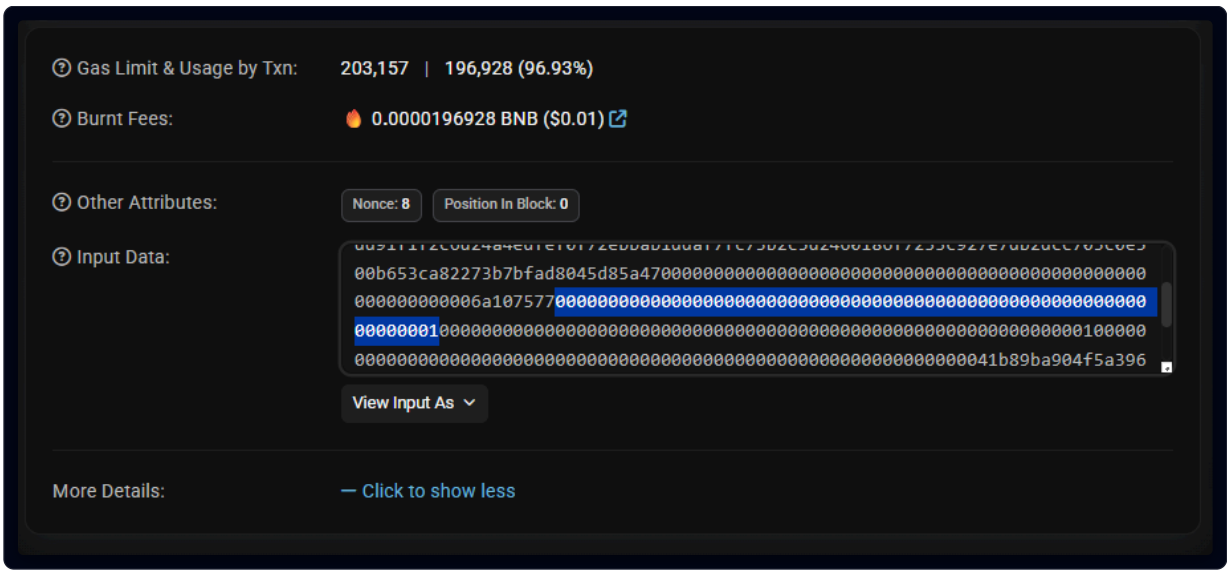
Slicing Slot 7: Notary Nonce



Slicing the next 32 bytes (64 hex characters): `0000000000000000000000000000000000000000000000000000000000000000`. This represents the notary's sequential sequence transaction count (nonce `0`), which prevents signature replay attacks.

STEP 10

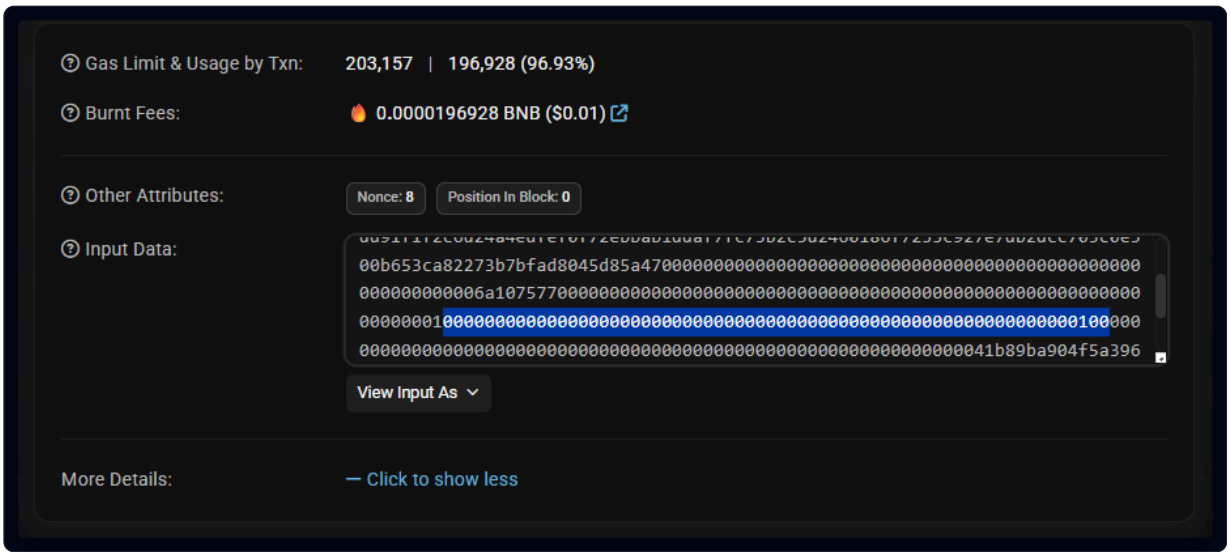
Slicing Slot 8: Dynamic Signature Offset Pointer



Slicing the next 32 bytes (64 hex characters): `0000000000000000000000000000000000000000000000000000000000000100`. In ABI encoding, dynamic arguments (like `bytes signature`) are placed at the end. This pointer says the signature payload starts exactly `0x100` bytes (256 decimal bytes) after the start of parameters.

STEP 11

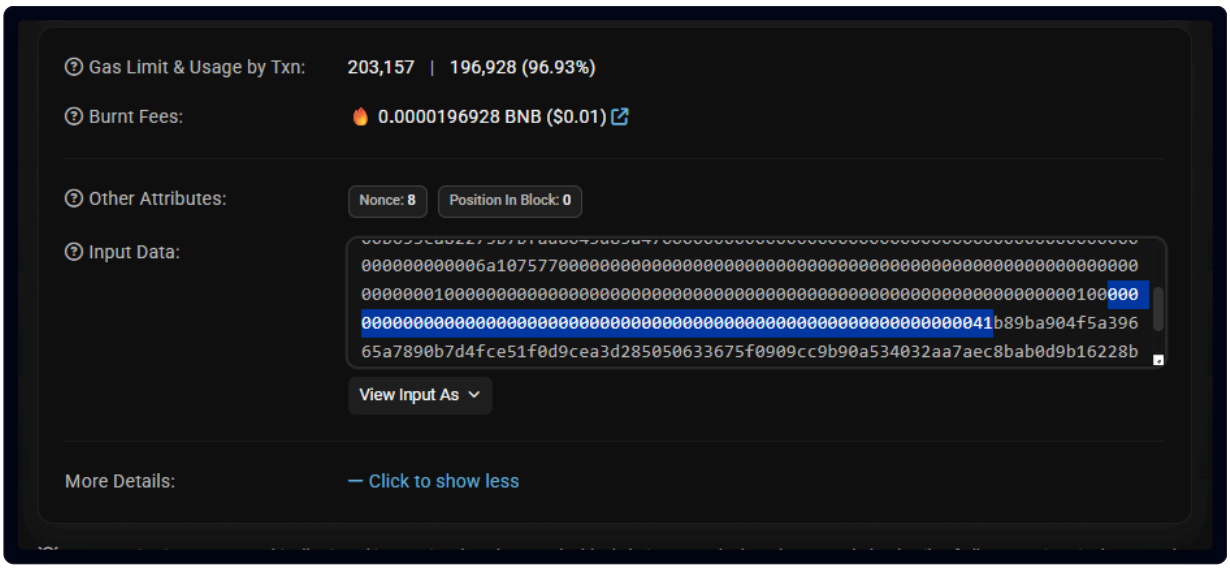
Visualizing Viewport Layout & Wrapped Alignments



This viewport shows how BscScan handles text wrapping. Looking at the alignment highlights the logical transitions between static offsets and dynamic array allocations within the default browser viewport.

STEP 12

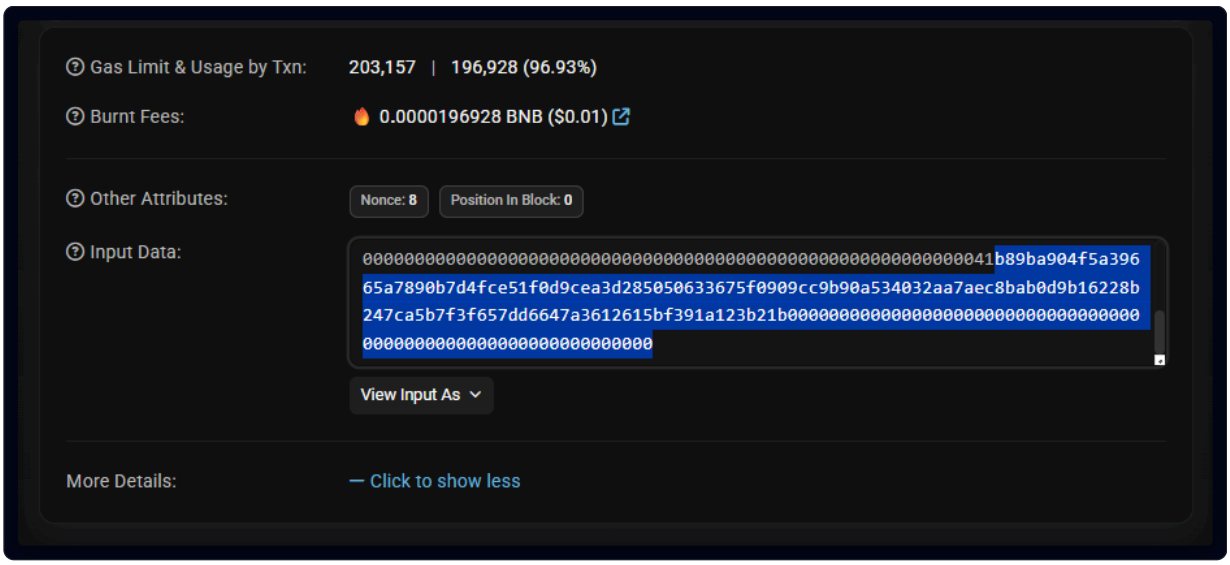
Slicing Slot 9: Dynamic Signature Byte Array Length



Slicing the next 32 bytes (64 hex characters): `0000000000000000000000000000000000000000000000000000000000000041`. This describes the length of the dynamic array. The hex value `0x41` represents **65 bytes** in decimal, which is the standard size of an ECDSA signature.

STEP 13

Slicing Slot 10: Cryptographic Notary Signature Bytes



Slicing the final byte payload:
`b89ba904f5a39665a7890b7d4fce51f0d9cea3d285050633675f0909cc9b90a534032aa7aec8bab0d9b16228b247ca5b7f3f657dd`
This is the 65-byte EIP-712 cryptographic signature (consisting of 32-byte `r`, 32-byte `s`, and 1-byte `v` values) proving authentic, authorized notary approval.

REFERENCE TABLE

EIP-712 Hex Data Layout Decoder

Use this comprehensive reference matrix to manually audit and slice the raw input bytes of the `recordAction` contract call:

Offset (Bytes)	Data Parameter	Hex Chunk Value (Example)	Decoded Meaning
`0x00`	Function Selector	`0xc7faf1fe`	Identifies the Solidity `recordAction()` method.
`0x04`	`docHash` (bytes32)	`f7d84bd8f2033d700b30c62b7f5354a4372859b82469060...`	SHA-256 unique digital fingerprint of the document.
`0x24`	`ownerAddress` (address)	`000000000000000000000000a82a6fcbb2db5979a6fcca83...`	Left-padded personal wallet address of the document owner.
`0x44`	`status` (uint8)	`0000000000000000000000000000000000000000000000000...`	`1` = Legally Approved on-chain.
`0x64`	`summaryHash` (bytes32)	`6505fc5cd9a3df581e8c23ddd91f1f2c6d24a4edfef0f72...`	Keccak-256 hash of the notary's summary text.
`0x84`	`rejectionReasonHash`	`c5d2460186f7233c927e7db2dcc703c0e500b653ca82273...`	`keccak256("")` constant. To satisfy Solidity's static bytes32 type layout, an approved document hashes a blank string, yielding this exact cryptographic constant.
`0xa4`	`timestamp` (uint256)	`0000000000000000000000000000000000000000000000000...`	Decoded UNIX timestamp of the notary signature.
`0xc4`	`nonce` (uint256)	`0000000000000000000000000000000000000000000000000...`	Notary sequential sequence ID to prevent replay attacks.
`0xe4`	`signature offset`	`0000000000000000000000000000000000000000000000000...`	Points to the starting byte index of signature bytes (Slot 9).

Offset (Bytes)	Data Parameter	Hex Chunk Value (Example)	Decoded Meaning
`0x104`	`signature length`	`0000000000000000000000000000000000000000000000000000...`	Signature length: `0x41` = 65 bytes.
`0x124`	`signature bytes`	`b89ba904f5a39665a7890b7d4fce51f0d9cea3d285050633...`	The raw ECDSA signature bytes packed: `r`, `s`, `v`.